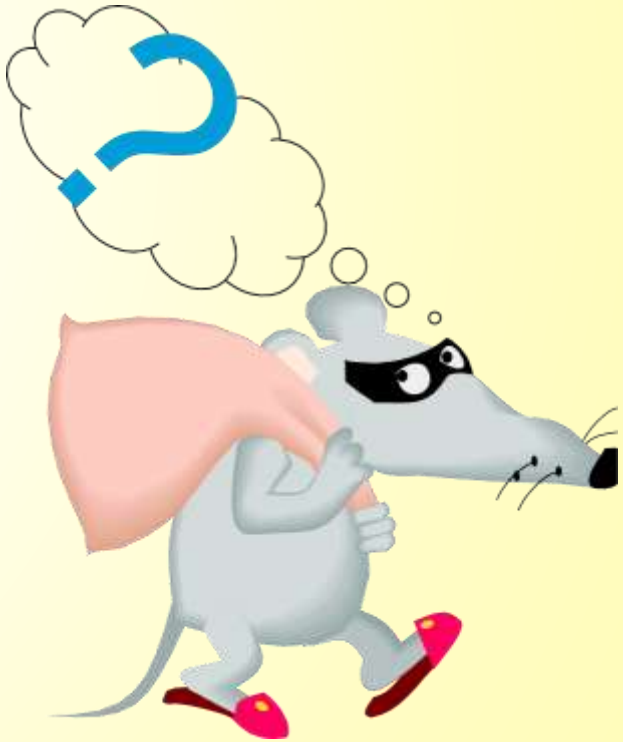
Contents:

- What is greedy approach?
- Greedy Algorithms and techniques.
- Knapsack problem.
- Knapsack Algorithm.
- Example of Knapsack problem.



Greedy Algorithms:

- Many real-world problems are optimization problems in that they attempt to find an optimal solution among many possible candidate solutions.
- An optimization problem is one in which you want to find, not just *a* solution, but the *best* solution
- A “greedy algorithm” sometimes works well for optimization problems
- A greedy algorithm works in phases. At each phase: You take the best you can get right now, without regard for future consequences. You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum.

Greedy Technique:

- Constructs a solution to an optimization problem piece by
- piece through a sequence of choices that are:

1. feasible, i.e. satisfying the constraints.

2. locally optimal (with respect to some neighborhood definition).

3. greedy (in terms of some measure).

Knapsack Problem:

- Given n objects each have a weight w_i and a value v_i , and given a knapsack of total capacity W. The problem is to pack the knapsack with these objects in order to maximize the total value of those objects packed without exceeding the knapsack's capacity.
- More formally, let x_i denote the fraction of the object i to be included in the knapsack, $0 \leq x_i \leq 1$, for $1 \leq i \leq n$. The problem is to find values for the x_i such that

$$\sum_{i=1}^{n} x_i w_i \leq W \quad \text{and} \quad \sum_{i=1}^{n} x_i v_i \quad \text{is maximized.}$$

The optimal Knapsack Algorithm:

This algorithm is for time complexity O(n lgn))

(1) Sort the n objects from large to small based on the ratios v_i/w_i . We assume the

arrays w[1..n] and v[1..n] store the respective weights and values after sorting.

(2) initialize array x[1..n] to zeros.

(3) weight = 0; i = 1

(4) while (i ≤ n and weight < W) do

(I) if weight + w[i] ≤ W then x[i] = 1

(II) else x[i] = (W – weight) / w[i]

(III) weight = weight + x[i] * w[i]

(IV) i++

Example of Knapsack problem:

- Let us consider that the capacity of the knapsack $W = 60$ and the list of provided items are shown in the following table –

Item	A	B	C	D
Profit	280	100	120	120
Weight	40	10	20	24
Ratio	7	10	6	5

- As the provided items are not sorted. After sorting, the items are as shown in the following table.

Item	B	A	C	D
Profit	100	280	120	120
Weight	10	40	20	24
Ratio	10	7	6	5

Solution:

- After sorting all the items, First all of ***B*** is chosen as weight of ***B*** is less than the capacity of the knapsack.
- Next, item ***A*** is chosen, as the available capacity of the knapsack is greater than the weight of ***A***.
- Now, ***C*** is chosen as the next item. However, the whole item cannot be chosen as the remaining capacity of the knapsack is less than the weight of ***C***.
- Hence, fraction of ***C*** (i.e. (60 − 50)/20) is chosen.

Cont.

- Now, the capacity of the Knapsack is equal to the selected items. Hence, no more item can be selected.
- The total weight of the selected items is

  10 + 40 + 20 * (10/20) = 60

  And the total profit is

  100 + 280 + 120 * (10/20) = 380 + 60 = 440
- This is the optimal solution. We cannot gain more profit selecting any different combination of items.

Item	A	B	C	D
Selected	1	1	0.5	0